A Datomic in beam-lisp

An immutable temporal database, built in the language it is written in

2026-08-21 · implemented, in-memory · status: feature-complete, awaiting a persistent substrate

1 What this is, from zero

A database where **nothing is ever updated or deleted**.

Changing an attribute does not overwrite anything. It appends two facts: a retraction of the old value and an assertion of the new one, both stamped with the transaction that made the change. The old value does not move to an archive — it stays exactly where it was, marked as no longer current.

Everything interesting follows from that single decision.

Because nothing is deleted	time travel is a filter over data that never went anywhere, not a feature that had to be built
Because a fact is self-contained	adding an attribute to one entity touches nothing else — there is no table shape to migrate
Because a database is a value	a query is a function of that value, not a request to a server, and holding one from an hour ago costs the same as holding an integer
Because queries are data	a program can build one with `conj` — no string assembly, no injection surface

This document explains each of those, shows the implementation, and ends with the measurements that should decide what storage substrate comes next.

1.1 Where it lives

```
priv/datom.bl              the public API — one namespace, the whole database
priv/datom/
  store.bl                 L1: the storage port (six methods)
  store-ets.bl             L0: an ETS implementation of it
  codec.bl                 order-preserving binary keys
  index.bl                 datoms, and the four indexes
  schema.bl                what an attribute means
  tx.bl                    transactions: tempids, upsert, net effect
  db.bl                    the database as a value
  conn.bl                  the one mutable thing
  time.bl                  as-of / since / history
  pull.bl                  reading trees out of the graph
  query/parse.bl           a query is data, so parsing is validation
  query/plan.bl            index selection and clause ordering
  query/engine.bl          the join
  query/agg.bl             aggregates
```

240 tests, 494 assertions in `test/bl/datom/`. Five executable tutorials in `examples/datom/`.

2 The datom

A fact has five parts:

```
[entity  attribute  value  transaction  added?]
[1       :person/name  "Ada"  1000001    true]
```

The first three are what you would expect. The last two are what make this different from a key/value store: the transaction says **when**, and `added?` says whether the fact **began or ended**.

That is the whole data model. There are no tables, no rows, no columns, and no schema-shaped storage — an entity is simply whichever datoms happen to share an id.

2.1 Why five fields and not three

With three fields you have a triple store, and you can answer "what is true?". With five you can also answer "what **was** true, when did it change, and who changed it?" — and you can answer them with the same query engine, because the extra fields are ordinary addressable positions.

```
[?e :person/age ?age]          ; what is true
[?e :person/age ?age ?tx]      ; and which transaction made it so
[?tx :audit/by ?who]           ; and who ran that transaction
```

2.2 Datom identity is all five fields

A subtle and expensive lesson. The index key originally carried `[e a v tx]`, on the reasoning that a datom is identified by what it says and when. But within **one** transaction a fact can be retracted and re-asserted, and those are two different datoms agreeing on all four fields.

Their keys were byte-identical, and the store is a keyed ordered set: whichever was written second silently replaced the first. Unrecoverable loss in a database whose entire premise is that nothing is lost — and invisible, because the surviving datom looked perfectly well-formed.

`op` now terminates every index key. Its position is deliberate: last, so it separates two otherwise-identical keys without disturbing sort order within a transaction.

3 The four indexes

The same datoms, sorted four ways. Which one serves a query is decided by which positions are already bound, and that decision is the planner's whole job.

index	sorted by	answers
EAVT	e a v tx op	everything about entity 42 → `entity`, `pull`
AEVT	a e v tx op	every `:person/name` in the database → column scans
AVET	a v e tx op	who is named Ada? → unique lookups, ranges
VAET	v a e tx op	what points **at** entity 42? → reverse references

AVET and VAET cost an extra write per datom, so they are maintained only for attributes that ask (`:db/index`, `:db/unique`, `:db.type/ref`).

3.1 Keys are binaries, not tuples

Erlang orders tuples by **arity first**, so `{1, 2}` sorts after `{1, 2, 3}` is false — but more importantly, a tuple key cannot support prefix range scans at all: the prefix has a different arity than the keys it should bound, so the scan returns nothing rather than a subset. That is not a performance problem, it is a correctness one, and it was found the hard way.

Binaries compare byte-wise, so a proper prefix sorts immediately before every key that extends it. `key-prefix` plus `prefix-end` (which appends `0xFF`) is therefore a valid scan window. This is the same technique FoundationDB's tuple layer uses.

3.2 The codec must project the language's order, not invent one

An order-preserving encoding is easy to get subtly wrong, and every mistake is silent. Four found by review, each of which corrupted an index without erroring:

integer overflow	encoding shifted by `2^63` without a range check, so `2^63` and `-2^63` collided at the same key
embedded NUL	strings terminate with `0x00`, so a value **containing** `0x00` could forge a component boundary: `"a\\0Pfoo"` encoded identically to two components. Escaped as `0x00 0xFF`, FoundationDB's solution, which preserves order because `0xFF` sorts above every byte a real continuation can begin with
float ordering	IEEE-754 negative floats sort backwards as raw bytes; needs a sign transform
unknown types	fall back to `term_to_binary`, which does **not** preserve logical order — so an attribute with no declared `:db/valueType` must be refused an AVET index rather than silently misordering it

4 The database as a value

`(db conn)` is O(1). It captures a **basis** — a transaction number — not data.

```
(def snapshot (db conn))
(transact! conn [...])          ; the world moves on
(q '[...] snapshot)             ; snapshot answers exactly as before
```

A held database value is immune to later transactions, forever. This is not snapshot isolation implemented at some cost; it is the trivial consequence of never mutating anything. The value is a small record holding the store, the schema, a basis, and a filter mode.

4.1 Reads are a filter over an append-only log

Because every historical datom is still in the indexes, a read must **decide what is currently true**:

1. scan the index for candidate datoms
2. discard anything after the basis
3. discard anything retracted at or before the basis

Step 3 is where a defect lived that no test caught for two waves. The original implementation asked the store "is this fact retracted?" for **every datom it filtered** — so reading N datoms cost N scans. Quadratic, and measurably: 300 entities took 57ms to filter against 5ms to scan raw, and a reviewer measured 45 **seconds** at 1000 values. Since the join calls this in a loop, every join was quadratic too.

It is now one pass: build a map of fact → latest retraction transaction, then filter. Linear.

4.2 A same-transaction retraction wins

Superseding a cardinality-one value emits the retraction of the old value at the same transaction that asserts the new one. A reader pairing them with `≤` treats the assertion as surviving its own retraction, and the attribute holds two values.

But `<` alone is not right either, because a transaction can retract **and then re-assert** the same value, and `tx` cannot order two operations inside one transaction.

The answer is not a better tiebreak. It is to **never write the contradiction**: a transaction records only its NET effect.

```
[[:db/add e :age 37] [:db/add e :age 38]]
  naively: assert 37, retract 36, retract 37, assert 38
  net:     retract 36, assert 38          (37 never existed)

[[:db/retract e :n 7] [:db/add e :n 7]]
  naively: retract 7, assert 7
  net:     nothing                        (nothing changed)
```

A transaction is atomic, so its intermediate states never existed to any observer. Recording them is not merely wasteful — it is **false**, history describing moments that did not happen.

5 Transactions

Transaction data is data. Maps and operation vectors mix freely:

```
(transact! conn
  [{:db/id -1 :person/name "Ada" :person/email "ada@example.com"}
   [:db/add ada :person/age 36]
   [:db/retract ada :person/nickname "A"]
   [:db/retractEntity old-id]
   [:db/cas ada :person/age 36 37]
   {:db/id :db/current-tx :audit/by "alice"}])
```

5.1 Tempids and upsert

A negative `:db/id` is a placeholder scoped to the transaction; the report says which real id it became. If a tempid asserts a `:db.unique/identity` value that some entity already holds, it **resolves to that entity** rather than creating a new one — which is what makes a transaction idempotent, and what makes "insert or update" a single operation with no race.

Two tempids claiming the same identity resolve to the **same** id, even when neither exists yet. Without that, transacting two maps with one email created two entities sharing a value declared unique: a database violating its own schema, silently.

5.2 The two forms that read

`:db/add` and `:db/retract` say exactly what they do. Two forms do not, and must consult the database to know what they mean:

`[:db/retractEntity e]` retracts every datom the entity holds **and every reference to it**. The incoming half is easy to forget and impossible to repair later — a retracted person whose `:person/friend` references survive leaves dangling pointers that every reader must defend against.

`[:db/cas e a old new]` asserts only if the current value is `old`. Without it, a read-modify-write silently loses a concurrent writer: both read 41, both write 42, one increment vanishes with no error anywhere. `old` of `nil` asserts the attribute is empty, which is "create exactly once".

5.3 The transaction is an entity

A transaction id is an opaque increasing integer. `(as-of db 1000042)` is answerable but not **askable**: nobody chose 1000042 and nobody can say what it means.

So every transaction records a fact about itself — `:db/txInstant` — as an ordinary datom. And because it is an entity, a caller can annotate it:

```
[:find ?who ?why
 :where [?e :account/balance 175 ?tx]
        [?tx :audit/by ?who]
        [?tx :audit/reason ?why]]
```

Fact → change → author, in one query. There is no audit table and no trigger: the audit trail is what the data model already was, once the transaction became addressable.

6 Datalog

A query is a vector.

```
[:find  ?name ?age
 :in    $ [?city ...]
 :where [?e :person/city ?city]
        [?e :person/name ?name]
        [?e :person/age ?age]
        [(> ?age 21)]]
```

6.1 A join is a shared variable

There is no `JOIN` keyword because there is no join **construction**. Two clauses that mention the same variable must agree on its value; that is the entire mechanism. Following a reference is the same thing:

```
[?e :person/name "Alan"]
[?e :person/friend ?f]        ; ?f bound as a VALUE here
[?f :person/name ?friend]     ; and as an ENTITY here
```

Direction is not a property of the data. Asking "who considers Ada a friend" changes only which position the variable appears in — no denormalization, no second index maintained by the application.

6.2 `:in`, and why four binding forms

`?x`	scalar	one value
`[?x ...]`	collection	one row per value — **this is how you write `IN`**
`[?x ?y]`	tuple	destructure one value
`[[?x ?y]]`	relation	a whole **table** passed as an argument

The collection form saves `WHERE city IN (...)` from being built by string assembly or run N times. The relation form is how a query joins against data the database does not hold: ids from another service, rows from a CSV, the results of a previous query.

6.3 Negation, disjunction, and their scoped forms

`:or` requires its branches to bind identical variables, because a row present in one branch and absent from another has no meaning. That rules out the common case — two branches reaching the same entity by different routes:

```
[:or-join [?person]                    ; only ?person escapes
  [?person :via/email ?x]              ; ?x and ?y are local
  [?person :via/phone ?y]]
```

`not-join` similarly scopes negation, so "this customer has no order" is sayable without introducing the order to the outer query.

6.4 The planner

Clauses are reordered so that each runs with as much already bound as possible. The heuristic is greedy and deliberately not a cost model: bind the cheapest clause first, where cheapness is how many positions are already known.

One defect here produced **wrong answers rather than slow ones**. Readiness was judged by whether every variable a clause mentioned was bound — but a function clause `[(* ?a 2) ?d]` **binds** `?d`, so it was never ready, fell to a source-order fallback, and any clause reading `?d` ran first against an unbound variable, discarding every row. The query answered `#{}` and answered correctly once the clauses were written in a different order, which is precisely the coupling a planner exists to remove.

7 Pull

Datalog returns tuples of scalars. An application usually wants a tree.

```
[:person/name
 {:person/friend [:person/name]}     ; nest through a reference
 {:person/manager ...}               ; recurse to any depth
 {:person/report 3}                  ; or a bounded depth
 :person/_friend                     ; REVERSE: who refers to me
 (default :person/nickname "none")
```

```
 (limit :person/friend 10)
 (:person/name :as :name)]
```

Because the pattern is data, the shape is a **parameter**. A UI knows which fields it is displaying, and that list is a pull pattern — the over-fetching problem that motivates GraphQL is answered by making the shape an argument.

7.1 Recursion and cycles

`...` descends until nothing new is reachable; an integer bounds the descent. A hierarchy of unknown depth — an org chart, a comment thread, a category tree — cannot be expressed by a fixed pattern at all.

Cycles are ordinary rather than exotic: mutual friendship is one, and so is any bidirectional edge. A revisited entity comes back as a bare `{:db/id N}`. The visited set tracks the path **from the root**, not a global set, because two siblings may legitimately reference the same entity and each should expand it.

7.2 What pull refuses

`entity` and `pull` both refuse a **history** view. A history database holds every value an attribute ever had, so folding it into a map would report an entity with three simultaneous ages — a value that existed at no point in time.

That is a category error, not a rare edge, so it fails loudly rather than returning a plausible-looking map describing a state that never was. A wrong answer nothing about which looks wrong is the worst outcome available.

8 Time

```
(as-of db t)     ; the world as it was  — inclusive of t
(since db t)     ; only what changed    — exclusive of t
(history db)     ; every datom, both kinds
```

`as-of` is O(1) to construct and copies nothing: the historical value shares the same store and differs only in its basis. Every query and every pull works against it unchanged. There is no `FOR SYSTEM_TIME`, no audit table, no separate API — **the query does not know it is looking at the past**.

`as-of` being inclusive and `since` exclusive means the two partition history exactly: every datom falls in one half or the other, never both, never neither. That property is asserted as a test, and it caught a defect where `history` cleared any existing bound and both views returned everything.

8.1 The actionable forms

```
(changes db e a)      ; every value the field has held, oldest first
(value-at db t e a)   ; what it held at t
(tx-of db e a v)      ; when did this become true?
```

`changes` had an ordering defect worth recording: sorting by `tx` alone left same-transaction datoms in **index** order — which is by value — so a balance history could read `175 began, then 250 ended`. The change described backwards. Within one transaction a retraction always precedes the assertion that supersedes it.

9 Schema

Schema is data, and it lives in the database. An attribute is an entity whose own attributes describe it:

```
(connect [{:db/ident :person/name
           :db/valueType :db.type/string
           :db/cardinality :db.cardinality/one}])
```

Written as ordinary datoms by an ordinary transaction, so `[?a :db/ident ?name]` answers, `pull` works on an attribute, and a schema change appears in history with a transaction and a timestamp.

9.1 The bootstrap

Schema datoms need a schema to validate against — the circularity every self-describing system meets somewhere. The answer is a fixed set of primordial attributes (`:db/ident`, `:db/valueType`, `:db/cardinality`, `:db/unique`, `:db/index`, `:db/doc`, `:db/txInstant`) installed into every schema. Everything else is ordinary data.

9.2 A known limitation, stated

Reads consult a schema **map** on the database value, not the datoms. The map is touched on every datom filter — cardinality decides whether a value collapses, index selection decides where a datom is written — so deriving it per read would be a serious regression.

The datoms are the record; the map is an index of it. Where they can disagree — a schema change mid-history — the map is the one that is current, so **a historical view reads old data through today's schema**. If an attribute's cardinality changed, a historical read interprets old datoms under the new rule. Filed as FEAT-010.

10 The storage port

Everything above L1 is pure logic over an ordered key/value space. The port is six methods:

```
(-get store k)            ; one key
(-range store lo hi)      ; ordered scan, [lo, hi)
(-put store k v)
(-delete store k)
(-update store k f)       ; atomic read-modify-write
(-commit store ops)       ; apply many operations, in order
```

A substrate must provide: **ordered keys**, **prefix range scans**, **atomic multi-key commit**, and **compare-and-swap**. Anything offering those four can carry this database unchanged.

10.1 The port is proven, not assumed

A port with one implementation is a hypothesis. `priv/datom/store-map.bl` is a second one — a sorted map in an atom, sharing no machinery with ETS: no tables, no process ownership, a different concurrency story. The entire database runs on it unchanged.

`test/bl/datom/conformance_test.bl` runs twenty tests against **both** implementations, and asserts they give identical answers. It earned its keep on its first run: the two stores disagreed about range bounds, because the second implementation had assumed half-open where the port specifies inclusive. That is the single easiest property for a backend author to get wrong, and it now fails loudly rather than silently returning one datom too few from every scan.

This suite is what makes the next backend safe. Run it against a new store and "it works" has a definition rather than being a hope.

10.2 What the ETS implementation does not provide

Stated rather than buried, because these are exactly what a real substrate is for:

- `-commit` is not atomic for a mixed put/delete batch. A process killed mid-batch leaves it half-applied, and a torn index is a corrupt database.
- `-update` is atomic only by single-writer discipline, not by the store.
- There is no durability at all. It is an in-memory development substrate.

The connection is likewise a record doing read-modify-write, so transactions are atomic **only under a single writer**. This is documented at the top of `conn.bl` rather than discovered later.

11 Measurements

11.1 Why counts, not milliseconds

Timings measure the machine as much as the code. While these were taken the host sat at load average 16 with 48 GB of swap in use, and two identical runs of the same benchmark differed by 2.2×.

Store **operations** are exact: they do not move with load, need no warm-up, and — the point — they are what decides what happens when the store stops being local. Every `-range` is one round trip against a networked backend.

11.2 Reads

Measured by wrapping the L1 port in a counting decorator, which is only possible because L1 is a protocol.

operation	N=10	N=40	shape
entity (point read)	1	1	constant
column scan	1	1	constant
two-clause join	2	2	constant
three-clause join	3	3	constant
pull (2 attributes)	3	3	constant
aggregate (count)	1	1	constant
as-of column scan	1	1	constant

Every read is constant in store operations. Reading the past costs exactly what reading the present costs, which is the db-as-value claim stated as a number.

11.3 The join was not always constant

It evaluated each clause once **per binding row**: 11 ranges at N=10, 41 at N=40. Against ETS a range is a local call and nobody notices. Against a networked store, a join over 1000 entities is **1001 round trips** — at 0.5 ms RTT, half a second of latency for data that fits in one packet.

Each pattern is now scanned once for the whole binding set and unified in memory. This is the single most important number here for choosing a backend, and it is asserted as a test so a regression shows up as a failing count rather than as a slow benchmark somebody has to interpret.

11.4 Range predicates read their window, not the column

A comparison on an AVET-indexed attribute lowers into scan bounds. Measured over 100 entities, counting datoms **returned** rather than ranges issued — a bounded scan and a full scan are both one range, so this is invisible in range counts:

query	answers	datoms read
indexed `(> ?a 95)`	4	5
indexed, no predicate	100	100
unindexed `(> ?a 95)`	4	100
two-sided window	4	6

The predicate still runs after the bounded scan. The bound narrows what is read; it does not replace the test — so an off-by-one in the bounds can only make a query slower, never wrong. That is the property that makes the optimisation safe to have, and the acceptance test asserts the indexed and unindexed paths give **identical** answers rather than asserting a speed.

11.5 Writes

entities	ranges	updates	commits
10	62	11	1
20	122	21	1
40	242	41	1

Linear, not constant — a transaction must check what each entity currently holds. Doubling the transaction doubles the work, which is the property that makes bulk loading possible at all. One commit per transaction, whatever its size.

11.6 Language-level costs found along the way

Profiling the database found three defects in beam-lisp itself, each affecting every program in the language:

`into` was a pessimisation	it took the transient path unconditionally, but transients pay a fixed setup cost plus `Process.get/put` per element — measured 13× slower at 500 elements, 88× at 2000, and quadratic when called per element in a reduce
`vec` built element by element	`mapv` and `filterv` are `(vec (map ...))`, so every eager traversal in the language paid a 3–7× penalty; building in one pass fixed it
the codec escaped NUL per byte	3.1 s to encode 800 datoms against 5 ms for the ETS inserts they became — 99% of write time. Replaced with binary BIFs and a `match` fast path for the common no-NUL case

12 What a review found that tests did not

Seven review rounds, 51 defects. The distribution is the interesting part: almost none were crashes.

silently wrong	wrong answers with no error: VAET consulted for non-reference attributes, duplicate clauses dropped, unbound predicates passing, a function clause scheduled before its input
silently lost	data that vanished: datom key collisions, an intermediate value written as history, a tempid leaked into durable data as a negative id
silently accepted	malformed input taken as valid: heterogeneous `:or` branches, empty combinators, non-integer entity ids, `:db/index` without a value type
stale claims	documentation describing behaviour the code no longer had — the most common category, and arguably the worst, because a header that gets believed is worse than no header

Two defects **cancelled each other into a passing test**: the datom key collision destroyed one of two datoms before the reader's `≤` bug could misjudge them. Fixing the writer turned the reader's bug red.

13 What is not implemented

Stated plainly, because a list of what a system does not do is more useful than a list of what it does.

- **Recursive rules** (`%`), Datomic's `[(ancestor ?a ?b)]` — the query language has no user-defined rules at all.
- **Multiple data sources** — one `$` only, so no cross-database joins.

- **Schema as-of.** A historical view reads through today's schema (FEAT-010).
- **Durability.** There is no persistent substrate yet — which is the point of the next section.
- **Multi-writer safety.** Single-writer discipline only.

14 Choosing a substrate

What the storage layer must provide is small and precise:

1. **Ordered keys**, compared byte-wise over binaries.
2. **Prefix range scans**, `[lo, hi)`, returning key/value pairs in order.
3. **Atomic multi-key commit** — a transaction's datoms reach every index together or not at all, because a datom present in EAVT but missing from AEVT is a corrupt database rather than a slow one.
4. **Compare-and-swap**, for id allocation and for `:db/cas`.

Everything else — the data model, the query engine, time travel — is already written and does not care.

14.1 What the measurements imply

Reads are O(1) in store operations, so a networked store costs one round trip per read **regardless of database size**. That is the good news, and it is the result of fixing the join.

Writes are O(n) in the size of the **transaction**, not the database. A bulk import of 10,000 entities is roughly 60,000 range reads today. Against a local store that is fine; against a networked one it is the thing to fix first, and the fix is known: a transaction knows all its entities up front, so the per-operation lookups can be hoisted into one range over the affected span.

14.2 The honest summary for the decision

The database is feature-complete in memory and does not depend on any property ETS has. The port is six methods wide, one of which (`-commit`) carries the only requirement that is hard: **atomicity across many keys**.

That single requirement is the axis the substrate choice should turn on. A store that provides it — whether an embedded engine reached through a NIF, a distributed one, or something assembled from parts — lets everything above L1 stay exactly as it is. A store that does not provide it forces the atomicity problem upward into code that is currently, and correctly, unaware of it.